

随机变量与条件概率实验

用精确分数核对条件分母

先修007。核心程序只用Python标准库，CPU、离线即可。data/inspection_counts.csv是原创1000件零件的完整有限总体，四格数值人为设计。默认实验按编号均匀抽一件；结果不是由真实样本估计的质量保证。

在016目录运行：

```
python experiment.py --output outputs
python test_experiment.py
```

脚本写results.json、prevalence_scan.csv、environment.json。Notebook重启内核后顺序运行，另写notebook_outputs；重跑覆盖对应目录的三个结果文件，不改数据。需要保留历史记录时，给脚本另一个--output目录。精确概率在JSON中同时保存分数字符串和展示小数。

A 手写事件和分母

先在纸上写出四格882、98、2、18。D为缺陷，F为标记。计算总件数、缺陷总数、标记总数、交集和并集，再分别算 $P(F|D)$ 、 $P(D|F)$ 、 $P(D|F^c)$ 。每个比例都写分母所代表的集合，不只写一个小数。

检查点A： $P(D)=1/50$ ， $P(F)=29/250$ ， $P(F|D)=9/10$ ， $P(D|F)=9/58$ ， $P(D|F^c)=1/442$ 。四种类别不是等概率样本点，不能直接把分母换成4。

B 让集合和分数对应纸笔对象

```
from fractions import Fraction
from experiment import load_table, OUTCOMES
model = load_table()
D = {(1, 0), (1, 1)}
F = {(0, 1), (1, 1)}
assert model.total == 1000
assert model.probability(D) == Fraction(1, 50)
assert model.conditional(F, D) == Fraction(9, 10)
assert model.conditional(D, F) == Fraction(9, 58)
```

集合里的元组顺序是(defective,flagged)，只能使用整数0或1。模型对同一事件中的重复组合只算一次，因为事件是集合。FiniteTable在内部复制计数，修改传入字典不会自动改变已建立的模型。API中的conditional(event,given)第二个参数就是分母条件。

C 从树的叶子重建Bayes

```
from experiment import bayes_flag
posterior = bayes_flag('1/50', '9/10', '1/10')
assert posterior['joint_defect_flag'] == Fraction(9, 500)
assert posterior['joint_good_flag'] == Fraction(49, 500)
assert posterior['flag_probability'] == Fraction(29, 250)
assert posterior['posterior_defect_given_flag'] == model.conditional(D, F)
```

检查点C：说明树上路径为何乘“边缘 $\times$ 条件”，两个标记叶子为何相加。接口要求两类先验均为正，让两种观察条件率有定义； $\pi=0$ 或 1 时拒绝。若两种标记率都为 0 ，标记条件质量为 0 ，也拒绝。比例输入用整数、Fraction或字符串，故意不接受二进制float。

D 改先验并区分模型假设

```
rare = bayes_flag('1/1000', '9/10', '1/10')
balanced = bayes_flag('1/2', '9/10', '1/10')
assert rare['posterior_defect_given_flag'] == Fraction(1, 112)
assert balanced['posterior_defect_given_flag'] == Fraction(9, 10)
assert not model.independent(D, F)
```

检查点D：读prevalence_scan.csv的五行。改变 π 时， r 与 q 被人为保持固定，这是一组假设模型，不是对真实部署环境的验证。说明为什么提高 $P(F|D)$ 不自动等于同幅提高 $P(D|F)$ ，以及为什么事件有关联不能单独确定因果方向。

E 抽样机制和多变量独立

```
from experiment import replacement_example, xor_world
draws = replacement_example()
assert draws['with_replacement']['second_given_first_defective'] == Fraction(2, 5)
assert draws['without_replacement']['second_given_first_defective'] == Fraction(1, 4)
xor = xor_world()
assert all(xor['pairwise_independent'].values())
assert xor['triple_111'] == 0
assert xor['product_of_three_marginals'] == Fraction(1, 8)
```

检查点E：先列出5件零件的25个放回有序对与20个不放回有序对，再从编号对直接数两次均缺陷。异或例子必须检查三对变量的全部四格，再看三者共同事件；不要用肉眼“随机感”代替联合概率。

F 两个观测相同的生成世界

```
from experiment import causal_world
assert causal_world('chain') == causal_world('common_cause')
chain_do = causal_world('chain', 1)
common_do = causal_world('common_cause', 1)
assert sum(row['mass'] for row in chain_do if row['y'] == 1) == 1
assert sum(row['mass'] for row in common_do if row['y'] == 1) == Fraction(1, 2)
```

检查点F：干预改变X的赋值规则，不是过滤U。两个世界的观察条件 $P(Y=1|X=1)$ 都为1，但给定规则下的干预结果不同。代码只证明这两个已定义世界的性质，没有从真实观测推断出机制。

G 主动检查无定义和非法输入

```
try:
    model.conditional(D, set())
except ValueError as error:
    assert 'positive conditioning mass' in str(error)
else:
    raise AssertionError('Zero-mass condition accepted')
```

再尝试负计数、布尔标签、缺失四格、重复CSV行、无法解析的比例、超出 $[0,1]$ 的比例，以及非法干预值。它们应明确拒绝。概率为零的条件集合即使非空也要拒绝，独立检查则用联合乘积式不需要做除法。

H 交付并解释证据边界

提交四格手算、每个条件分母、完整树路径、五组先验扫描、抽样机制说明，以及两个世界的观测/干预对照。测试含100个独立枚举总体，每个总体检查256对事件；重复运行和Notebook三文件一致。

核心计算只用精确Fraction，最终小数是展示近似。本教材随附Notebook在新Python进程中的真实IPython InProcessKernel顺序执行，未验证Jupyter浏览器、跨进程socket或全新Anaconda环境。计数模型和因果世界都是教学合成设定，不证明真实设备性能、总体独立性或现实因果关系。